

Implementation of Kalman Filter and Deep-Learning Based Trackers

Benjamin Luo

Department of Management Science and Engineering
University of Waterloo
Waterloo, Canada
b33luo@uwaterloo.ca

Overview—This report overviews the implementation and evaluation of Kalman Filter and Deep-Learning based tracking algorithms on the VOT 2018 ST dataset, with a focus on the butterfly and helicopter datasets.

I. METHODOLOGY

A. Kalman filter tracker

The Kalman filter (KF) tracker is a classical method used to estimate states between consecutive frames. The state vector is in the form $[x, y, w, h, xv, yv, wv, hv]$ where x and y are the centerpoint of the bounding box (bbox), and w and h are the width and height, respectively. The ‘v’ suffix estimates the velocity of each component. The implementation follows:

- (1) **Initialize** the state and covariance matrices
- (2) **Predict** the state using the state transition matrix (which tracks system dynamics) and the previous state estimate
- (3) **Extract measurement values** (x, y, w, h) from the current frame and compute the bbox
- (4) **Update the state** estimate based on the measurement matrix (which tracks the relationship between the state and measurements)
- (5) Iterate for each frame in the sequence

For the initialization, I provide the ground truth bbox to the model. Alternate solutions could include empirically identifying the initial boundary, or using an object detector model.

B. Deep-learning based tracker

There are various deep-learning (DL) based trackers which leverage deep neural networks for object detection. Figure 1 shows a breakdown of the tracking models used on the VOT-18 datasets, of which the majority are DL based. These DL models generally perform better than classical models in terms of accuracy as they can better detect variations in lighting, pose, and other environmental factors. For example, they are better equipped to handle occlusion and non-rigid objects. However, they also come with higher overhead as models must be trained and are also highly computational.

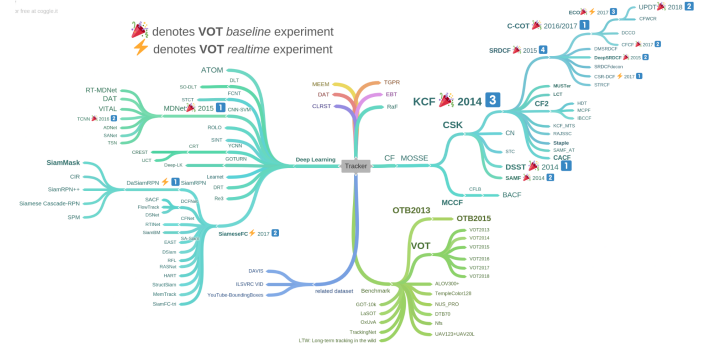


Fig. 1. A breakdown of popular models tested on the VOT2018 dataset^[1]

For model selection, I attempted to install and run Siamese RPN, SiamFC, SiamNanoTrack, SiamFCPP, PrDiMP, and YOLOv8. These were selected as they were (1) open-source with (2) pretrained weights, (3) many GitHub stars, and (4) performed well based on the VOT18 model comparison^[1]. However, I encountered dependency issues for most models due to their lack of CPU support (the majority of state-of-the-art models use CUDA by default).

I settled on using YOLOv8 (You Only Look Once) through the ultralytics library^[2]. This is a real-time object detection system that can detect objects in a single pass. Its architecture uses a convolutional neural network to predict bounding boxes and class probabilities given an image frame. For each generation of models, it offers 5 pretrained weight files (table 1) that were trained on the COCO (Common Objects in Context) dataset. I compare the nano and x-large model as they are the most different and demonstrate the complexity v accuracy tradeoff.

Table 1: YOLOv8 pretrained weight files^[1]

Model	Parameters (M)	FLOPs (B)	Size (MB)
YOLOv8n	3.2	8.7	6.25
YOLOv8s	11.2	28.6	21.54
YOLOv8m	25.9	78.9	49.72
YOLOv8l	43.7	165.2	83.73
YOLOv8x	68.2	257.8	130.55

C. Data and quantitative evaluation

The Visual Object Tracking (VOT) 2018 dataset is used for evaluation. It consists of 35 video sequences, such as a butterfly flying over flowers and a helicopter flying into the horizon. This analysis will focus on the butterfly (n=151 frames) and the helicopter (n=708 frames) sequences in particular (figure 3).

Quantitative measurement is done based on the intersection over union (IoU) of the predicted bounding box and the ground truth bounding box. It is calculated using the x and y coordinates of each bounding box where the intersection area is divided by the union area. IoU scores are assigned per frame and will be aggregated for each approach as both mean and median values.

Computational efficiency is another metric of interest as DL trackers are more computational than most classical trackers, and this trade-off between cost and accuracy must be considered for practical implementations. Total runtime (s.) and frames per second (FPS) will be collected for each model and sequence. Runtime is estimated by the Jupyter notebook's runtime for the cell containing the tracking logic and includes computation of the IoU. Only 1 trial is recorded for each run as empirical observations show runtime is consistent ($\pm 1s.$) between executions. FPS is calculated by dividing the total number of frames over the total runtime.

D. Qualitative evaluation

Each VOT-18 sequence presents challenges in video processing, including occlusion, deformation, and background noise. Frames that exhibit these traits will be identified and empirically analyzed for model performance. Analysis will be done on what traits lead to the tracker losing the target, as well as the geometry of the bounding box relative to the ground truth.

The butterfly sequence shows a moving butterfly with flowers of similar color in the background. The butterfly also appears to change color due to the lighting, as well as deforms considerably when it flaps its wings. A portion of the clip shows the butterfly partially occluded into the upper corner of the image.

The helicopter sequence shows a large red helicopter taking off into the horizon. Unlike the previous clip, the camera is moving and there is significant noise in the background through the movement of people. This is also a very long sequence with 708 frames, so it represents a long-term tracking problem. Strangely, the ground truth is reported as the body of the helicopter instead of the entire helicopter; excluding the rotating blades and tail. Finally, the sequence presents a unique challenge as the helicopter fades into the horizon. This is significant because the background objects become larger than the helicopter and may confuse the tracking algorithm's targeting system. The helicopter also rotates which deforms its shape and alters its lighting.

II. RESULTS

Table 2: Quantitative comparison between KF and DL trackers

Model	Sequence	Mean IoU (%)	Median IoU (%)	Runtime (s.)	FPS
Kalman (cv2)	Butterfly	52	49	0.10	1510
	Helicopter	62	63	0.50	1416
YOLOv8s	Butterfly	8	0	5.70	26
	Helicopter	9	9	21.50	32
YOLOv8x	Butterfly	4	4	47.10	3
	Helicopter	9	10	177.50	4

E. Kalman filter tracker

Table 2 shows a summary of the quantitative analysis done on all models. The KF tracker significantly outperformed the DL tracker for both sequences based on IoU and computation metrics.

Qualitatively the KF tracker also performed exceptionally with almost every predicted bounding box intersecting with the ground truth bounding box ($\text{IoU} > 0\%$) as seen in figure 2.

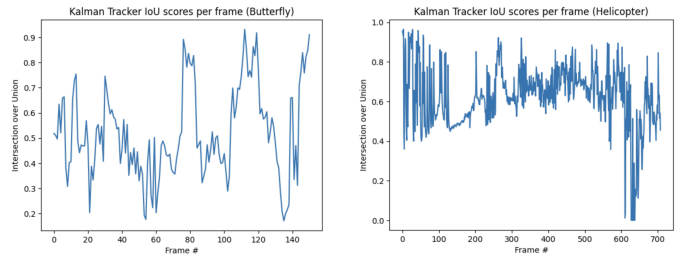


Fig. 2. KF tracker IoU scores per frame (higher IoU is better)

However, the predicted bbox (green) did appear to slightly lag behind the ground truth bbox (red), especially for sudden movements as seen in figure 3. Also note that the ground truth includes rotation whereas the predicted bbox only has translation.

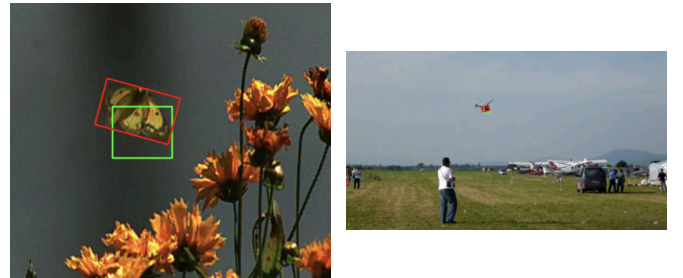


Fig. 3. KF filter applied to the butterfly (left) and helicopter (right) sequences (green is predicted bbox, red is ground truth bbox).

F. Deep-learning based tracker (YOLOv8)

Both YOLOv8s and YOLOv8x pretrained models were tested and performed worse than the KF tracker. Table 2 reports the mean IoU as between 4% to 9% for all sequences, and the computation was also dramatically slower with the small model running at 26-32 FPS and the x-large model running at 3-4 FPS.

When compared to each other, both models share similar results and shortcomings. In terms of accuracy, they perform about the same (table 2), but the x-large model runs about 8x slower based on the FPS.

Figure 4 shows IoU across frames. The choppy graphs indicate the tracking was frequently lost. In fact, the YOLOv8n tracker failed to establish the butterfly from the very start of the sequence (figure 6).

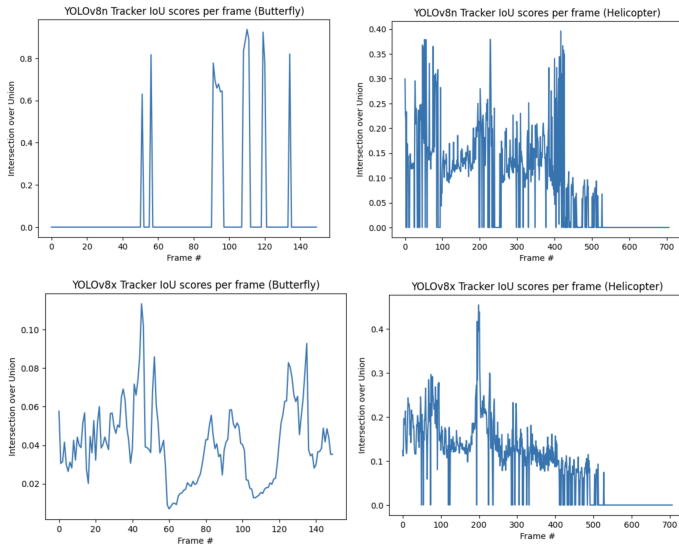


Fig. 4. YOLOv8n (top) and YOLOv8x (bottom) trackers' IoU scores per frame (higher IoU is better)

When objects were successfully detected ($\text{IoU} > 0\%$), the predicted bbox was often excessively large (figure 5). In general, the models were fairly conservative and attempted to capture the entirety of the object. For example, the helicopter was tracked with both the blades and tail.



Fig. 5. YOLOv8 predicted bbox overestimating the object size

Figure 6 demonstrates instances where YOLOv8 failed to detect the foreground object. For the butterfly sequence, the object detection likely confused the butterfly as a flower, and instead focused on the stem, which appeared visually distinct from the remainder of the image. For the helicopter sequence, the helicopter became increasingly small as it flew into the horizon, and the model instead prioritized the moving person who appeared larger and exhibited greater motion flow.



Fig. 6. YOLOv8 failing to detect the foreground object

III. DISCUSSION

G. Exploring YOLOv8's performance

A huge contributor to the low performance is that DL models with pretrained weights may rely on fine-tuning for achieving greater results. The YOLOv8 model in particular was trained on the COCO dataset which only has 80 object categories, so it may not be familiar with butterflies and helicopters. Furthermore, YOLOv8 is designed to be a high-performance model compared to other state-of-the-art trackers so it may sacrifice accuracy for performance, especially given the relatively small pretrained weights (6-130MB) compared to offerings from other models that typically had weights between 300-400MB. Only the base hyperparameters were used, but a sensitivity analysis was briefly conducted to test the effect of altering each hyperparameter and these changes were unable to noticeably affect the performance. This supports the claim that the model weights are the main bottleneck and must be fine-tuned.

A suboptimal implementation may also be to blame for the poor accuracy. YOLOv8 returns a list of object classes and their probabilities so it is important to carefully select which object to track. Under the current implementation, if the object of interest is not detected with a sufficiently high accuracy, then the largest bbox is selected. This may explain the false detections seen in figure 6.

In terms of overestimating the object size (figure 5), it is difficult to fault the model when it comes to the helicopter. The blades and tail are technically part of the helicopter object; they are connected and exhibit high motion flow. However, the ground truth label is only interested in the main body.

It is also quite surprising that the YOLOv8 model lost detections after successfully locking onto the object. It is technically a hybrid model that uses either BOTSORT or ByteTrack to track the object after the DL model identifies the object. The discontinuous tracking is likely attributed to the object being lost between consecutive frames, which is subsequently caused by the high deformation of the objects. For example, figure 7 shows the butterfly occlude and deform significantly, which may confuse the object detector. Once the detection is lost, then the tracking is reset.



Fig. 7. Occlusion and deformation in the butterfly sequence

Future work could see fine-tuning of the YOLOv8 model on excerpts of the VOT-18 dataset and data from previous years. It would also be interesting to assess a more popular DL-based tracker such as one of the Siamese models (figure 1).

H. Exploring KF performance

The KF filter performed very well in all quantitative assessments (IoU, computation). The bbox was consistently an appropriate size and always intersected with the ground truth bbox. However, a major fault is that the ground truth bbox includes rotation, as seen in figure 7. The KF bbox purely relies on translation so it is incredibly rare to achieve a perfect IoU (100%).

No re-initialization was necessary as there was no full occlusion in either sequence and the tracking was never lost. As such, there were technically no detections used in the KF tracker (the first bbox was initialized using ground truth instead of an object detector). Incorporating a detector, such as YOLO would drastically increase runtime, especially as sampling frequency increases. This is very apparent in the YOLO tracker where tracking was done on every frame; leading to a significantly longer runtime (table 2).

In terms of parameter selection, the process noise ($Q=1e-4$) and measurement noise ($R=1e-3$) were adjusted impromptu but did not lead to considerable changes in the output, similar to hyperparameters in the YOLOv8 model. In theory:

- Q represents uncertainty in the motion model. Increasing Q will make the model more responsive to unpredictable motions, whereas decreasing Q leads to smoother motion but higher lag (figure 3)
- R represents confidence in detections. Increasing R indicates more trust for the predictions, whereas

decreasing R will give more weight to the measurement matrix

In this case, $Q < R$ to give more weight to the measurements than the motion model. This is more favorable because the motion is non-linear in both sequences, especially for the butterfly sequence.

Figure 3 hints that the KF model lags behind the ground truth, particularly for consecutive frames with considerable motion. This is more prevalent in the butterfly sequence as butterflies flap their wings rapidly which leads to large discontinuities between adjacent frames – a weakness of conventional trackers.

It is also important to note that the KF model was initialized with the ground truth bbox instead of through an object detector. This advantage may propagate across frames and artificially improve model accuracy.

I. Comparative analysis

The KF tracker outperformed the YOLOv8 tracker both quantitatively and qualitatively. However, this may be largely attributed to the implementation as the YOLOv8 models were not fine-tuned. Nonetheless, the KF tracker still significantly outperforms the YOLOv8 trackers in terms of performance by approximately 8x.

The KF tracker was also able to better overcome background noise in both sequences. That is, it distinguished the butterfly from the flowers, and the helicopter from the moving observers, whereas the YOLOv8 tracker struggled in both cases. Though, it is important to consider that the objects did not undergo full occlusion so it was possible to trace them frame-to-frame - otherwise DL models would typically hold an advantage. The objects were also highly textured which makes classical approaches like the KF tracker appropriate.

Theoretically, DL models should perform better than classical methods due to their increased computational power and pattern recognition abilities, especially in the following conditions:

- Full object occlusion
- Low-texture environments (ex. glass)
- Object deformation (ex. flying butterflies)
- Non-linear motion (ex. falling leaves)
- Object detection in noisy backgrounds (ex. traffic)
- Large displacements between frames (ex. race cars)

Few of these conditions actually applied in this case as the butterfly and helicopter were relatively easy to distinguish from the background and could be smoothly tracked by classical models

IV. REFERENCES

- [1] https://github.com/foolwood/benchmark_results
- [2] <https://github.com/ultralytics/ultralytics>